Adventure Quest

A Factor Graph-Driven Interactive Narrative RPG

Group Report

ID	Name	Email
1731883642	Samiul Alim	samiul.alim@northsouth.edu
2011675042	Margia Hossain Mukti	margia.mukti@northsouth.edu
2031157642	Ashmit Ebne Nur	ashmit.nur@northsouth.edu
2415267650	Nusrat Jahan	nusrat.jahan50.241@northsouth.edu
2212439042	Nazia Tasnim	nazia.tasnim@northsouth.edu

Course: CSE440

Institution: North South University

Date: 27 April 2026

Adventure Quest: A Factor Graph-Driven Interactive Narrative RPG

***Abstract*—Adventure Quest is a browser-based interactive narrative role-playing game developed as part of the CSE440 curriculum at North South University. The project employs a factor graph as its central decision engine, replacing static branching dialogue trees with a probabilistic belief-propagation model. There are seven state variables, courage, trust, karma, wisdom, chaos, health, and wealth, which are updated after every player choice, and their evolved beliefs determine the scenes presented, the choices available and ultimately one of five primary endings plus an emergent sixth path. The application is implemented as a static single-page web application comprising three files: `Main.html`, `src/main.js` and `src/styles/game.css`, and requires no build pipeline, framework, or external API. The live factor graph is rendered on an HTML5 Canvas and remains visible to the player throughout the session, making the probabilistic engine a first-class narrative artefact rather than a hidden backend system. Verification was conducted through JavaScript syntax validation, a Playwright browser harness, screenshot reviews and a complete twelve-turn end-to-end playthrough that confirmed the ending modal fires correctly upon loop completion.**

Project Type: Browser-based 2D game
Technology: HTML5, CSS3, Vanilla JavaScript, Canvas, Web Audio API

I. Project Background

Adventure Quest originated from two foundational planning documents: a development plan and a visual design bible. The development plan defined the project as a dark fantasy role-playing game in which player decisions are processed through a factor graph. Rather than routing decisions through a conventional decision tree, the plan specified that every choice would update probabilistic beliefs associated with core state variables, allowing the game's narrative trajectory to emerge from the accumulation of player behaviour over twelve turns.

The visual design bible established the *Ember and Ash* identity that governs the user interface: dark base surfaces, glowing ember and gold accents, manuscript-inspired typography, ornamental panel borders, animated character silhouettes, scene strips, and a three-panel layout. These aesthetic choices were carried through to the final implementation without modification, ensuring visual consistency between the original specification and the delivered product.

A defining educational goal of the project is to make the factor graph visible and interpretable to the player. Accordingly, the left panel of the interface is dedicated to the live belief engine, the centre panel presents the story and available choices, and the right panel displays the player's arcane measures, inventory, and turn log. This arrangement allows a player or an evaluator to observe how each decision propagates through the model in real time.

A. Objectives

The project pursued five primary objectives. First, to deliver a fully playable two-dimensional browser game with a polished dark fantasy interface. Second, to implement a genuine factor graph engine rather than a decorative graph animation, ensuring that the mathematical model drives gameplay outcomes. Third, to expose probabilistic state changes through choice badges, graph edges, and animated stat bars so that the belief-propagation process remains legible to the player. Fourth, to support a complete twelve-turn gameplay loop terminating in an ending selected from the final belief state. Fifth, to maintain a static file structure deployable locally without any framework, build pipeline, or external service.

II. Requirements Analysis

A structured review of project requirements was conducted against the implemented system to confirm that all functional and non-functional goals were satisfied.

The interactive narrative loop, which required scene rendering, three choices per turn, and state-dependent narrative output, was fully implemented. The factor graph decision engine, demanding variables, factors, weights, messages, and belief-update logic, was realised through the `FactorGraph` class. Live graph visualisation, specified as a canvas-based renderer with animated message and edge behaviour, was delivered by the `GraphRenderer` component. The player state system tracked seven beliefs alongside inventory data, faction values, a turn log, act, location, and ending state.

Responsive user interface behaviour was implemented and subsequently refined to correct a viewport-overflow issue. API independence was achieved by removing remote API controls present in earlier design drafts; the game now relies entirely on a local narrative engine, eliminating the need for API keys, network access, or backend services. Testing evidence, including Playwright screenshots, JSON state snapshots, and end-to-end ending verification, was produced and retained under the `output/` directory.

III. System Architecture

The application is intentionally lightweight. `Main.html` provides the document structure and all necessary DOM anchors, `game.css` defines the complete visual language and responsive behaviour, and `main.js` implements the entire runtime. This deliberate separation of concerns improves maintainability while preserving the goal of a static, easily deployable project that requires only a modern browser. Table I

summarises the three runtime files and their approximate line counts.

TABLE I
RUNTIME FILE SUMMARY

File	Responsibility	Lines
`Main.html`	Page shell, semantic panels, controls, modal, script and style references	121
`game.css`	Theme variables, layout, responsive behaviour, panels, cards, animation styling	1,017
`main.js`	Game state, factor graph, narrative engine, canvas renderer, particles, sound, UI events	1,138

A. Runtime Components

The application runtime comprises seven principal components. The `FactorGraph` component stores variable priors, factor definitions, message tables, factor potentials, and the belief-propagation routine. `GameState` tracks the current turn, act, and location, as well as inventory contents, faction standings, narrative history, the current scene tag, and ending conditions. `NarrativeEngine` generates the opening scene and all subsequent scenes from current graph beliefs, producing both story text and choice descriptors. `GraphRenderer` draws variable nodes, factor nodes, weighted edges, belief labels, and animated graph activity on the HTML5 Canvas. `ParticleSystem` renders atmospheric ash particles across the full game screen. `SoundSystem` produces procedural user-interface and ending sound effects through the Web Audio API. A suite of UI functions manages rendering of narrative text, choice cards, stat bars, inventory listings, faction displays, ending sequences, and control events.

B. Data Flow

The data flow through the system follows a deterministic seven-step cycle that repeats on every player action. When the player selects a choice card, `GameState` applies the associated belief deltas and identifies the source factor. `FactorGraph` then updates variable priors and factor weights before executing the belief-propagation pass. `GraphRenderer` animates the updated graph state. `NarrativeEngine` uses the resulting beliefs to generate the next scene and a fresh set of three choices. The user interface refreshes the narrative panel, choice cards, animated stat bars, inventory listing, faction display and turn log. After turn 12, `GameState` evaluates the final belief values, selects the appropriate ending path and opens the ending modal.

IV. FACTOR GRAPH MODEL

The mathematical model is represented as a bipartite graph consisting of variable nodes and factor nodes. Variable nodes represent aspects of player and world state whose values evolve throughout the game. Factor nodes represent categories of narrative events; each factor connects to a subset of the variables it influences. When the player makes a choice, that choice contributes signed belief deltas to a designated set of variables and charges a source factor. Beliefs are then recomputed through a sum-product-style message-passing algorithm, and results are immediately reflected in the canvas visualisation and stat bars.

A. Variable Nodes

The model defines seven variable nodes, each capturing a distinct dimension of game state. *Courage* measures the player character's willingness to face danger. *Trust* quantifies reputation and credibility among non-player characters. *Karma* reflects the moral alignment of cumulative decisions. *Wisdom* represents accumulated knowledge and strategic insight. *Chaos* captures instability and danger in the world state. *Health* tracks the physical condition of the player character. *Wealth* measures available resources and supplies. Each variable is maintained as a normalised two-state belief in the interval $[0, 1]$.

B. Factor Nodes

Five factor nodes connect subsets of the variables. *CombatFactor* connects {courage, health, chaos}, reflecting the interdependence of physical risk, condition, and world volatility. *DialogueFactor* connects {trust, karma, wisdom}, capturing the relationship between social credibility, moral standing, and insight. *ExplorationFactor* connects {wisdom, courage, wealth}, representing the combined demands of venturing into unknown territory. *MoralFactor* connects {karma, trust, chaos}, encoding the systemic consequences of ethical choices. *ResourceFactor* connects {wealth, health, chaos}, reflecting trade-offs between material resources, physical wellbeing, and environmental danger.

Factor potentials are computed from the factor charge assigned by the chosen action, the weights associated with connected variables, and special-case adjustments for moral and combat pressure scenarios. The resulting belief propagation produces an updated posterior for each variable, which is consumed by `NarrativeEngine` to select the next scene template and by the ending-evaluation logic to determine the final outcome path.

V. USER INTERFACE AND VISUAL DESIGN

The interface adheres throughout to the *Ember and Ash* identity. Dark base surfaces are combined with ember and gold accents, manuscript-inspired typography, ornamental borders, and glowing canvas elements. The layout is divided into three primary regions: the belief engine panel, the narrative panel, and the state HUD, which are simultaneously visible on desktop screens. This arrangement allows a player to observe the probability model updating in real time while reading the story and selecting choices, reinforcing the educational intent of exposing the factor graph as an interactive artefact.

A. Major UI Elements

The interface comprises eight primary visual elements. A header bar displays the game title, current turn counter, act indicator, and a world event chip. The canvas factor graph

Fig. 1. Verified desktop interface showing the belief engine, narrative panel, and state HUD.

renders variable nodes, factor nodes, coloured weighted edges, belief labels, and a legend. A scene strip provides generated SVG landscape atmosphere. A character stage presents SVG silhouettes alongside a dialogue bubble. The narrative text area applies drop-cap styling and a typewriter reveal animation to generated story text. Choice cards display available actions annotated with factor deltas and outcome tags. The Arcane Measures panel shows animated stat bars for all seven belief variables, the current inventory, and a scrollable turn log. The ending modal displays the final narrative copy and a set of belief bars representing the terminal belief state.

VI. Implementation Details

A. Gameplay Loop

The game begins at Crossroads Village with an opening scene generated by `NarrativeEngine` from initial belief values. Each choice contains five components: the choice text, a factor hint, a source factor identifier, a scene tag, and a predicted outcome label. When selected, the system applies belief deltas, updates faction and inventory data, advances the turn counter, and invokes `NarrativeEngine` for the subsequent scene. This cycle repeats until the turn counter exceeds 12, at which point `GameState` evaluates final beliefs and opens the ending modal.

B. Local Narrative Generation

The original project plan described an external narrative API as the source of scene text and choice generation. During implementation, the remote API path and sidebar API bridge controls were removed from the codebase entirely. The project now relies on a deterministic local narrative engine, which improves classroom portability by eliminating the need for API keys, network calls, or backend services. This change was verified by confirming that no Claude, Codex, or API bridge identifiers remain in the active HTML, CSS, or JavaScript source files.

C. Responsive Behaviour

On desktop screens the application presents the full three-panel layout simultaneously. On narrower screens the panels reflow into a single-column vertical stack, and the graph panel becomes collapsible. A subsequent patch addressed a viewport-overflow issue in which typewriter text growth caused the page to scroll rather than using internal scrolling. The fix constrains the application shell to the viewport, ensuring that `window.scrollY` remains at zero during normal desktop gameplay.

D. Accessibility and Usability

Semantic HTML regions and ARIA labels are applied to the graph canvas, narrative area, stats panel, and ending modal. Keyboard focus states are defined for all interactive controls. A `prefers reduced motion` media query disables or reduces animations for users who have configured that preference. Choice-card readability was improved with brighter text, stronger contrast ratios, and bolder factor delta badges. The application also exposes `render game to text` and `advanceTime` helpers for automated test harnesses.

VII. Testing and Verification

The project includes a suite of verification artefacts retained under the `output/` directory, documenting syntax validation, browser harness execution, screenshot reviews, API-removal confirmation, readability improvements, scroll-behaviour corrections, and a complete end-to-end run through the twelve-turn gameplay loop. Table II summarises the verification activities and their outcomes.

TABLE II
Verification Activity Summary

Activity	Method / Artefact	Result
JS syntax check	`node --check src/main.js`	Pass
Web-game harness	Playwright; turn 1→2, no error files	Pass
End-to-end run	12 turns clicked; ending modal confirmed	Pass
Scroll fix	`window.scrollY`= 0 on desktop	Pass
API removal	No Claude/Codex identifiers in source	Pass
Readability review	Visual check; saved to `choices.png`	Pass

A. Sample End-State Evidence

One recorded end-to-end playthrough reached turn 13 in ending mode at the location identified as *The Final Chamber*. The terminal belief values were: courage 0.98, trust 0.99, karma 0.97, wisdom 1.00, chaos 0.48, health 0.76, and wealth 0.78. These values collectively satisfy the conditions for a high-karma, high-trust ending path and confirm that all seven state variables persist and accumulate correctly across the full twelve-turn gameplay loop.

VIII. Limitations and Future Work

Adventure Quest is complete as a fully playable local prototype satisfying all objectives defined in the project plan. Nevertheless, several extensions have been identified for future development cycles.

Formal unit tests for the `FactorGraph` component, covering message calculations, factor potential evaluations, and ending-selection logic, are absent and represent the most significant gap in verification coverage. Game progress is not persisted between sessions; `localStorage` support would allow a player to resume from a saved state. An optional export function for the final ending narrative and belief state would facilitate sharing and academic submission. Narrative variety could be expanded by introducing additional location-specific scene templates. Further character-specific dialogue animations would enrich visual storytelling without modifying the underlying graph model. Finally, a concise `README` and deployment instructions would reduce submission friction.

IX. Conclusion

Adventure Quest successfully integrates interactive fiction, probabilistic modelling, and polished browser interface design into a cohesive single-page application. The factor graph is not merely an internal mechanism; it is presented as a visible, animated, and interpretable component of the player experience, bridging the gap between computational reasoning and narrative engagement. Every choice the player makes produces an observable change in the belief engine, providing an intuitive demonstration of how probabilistic inference can drive game-state evolution.

The final implementation satisfies all major goals: a fully playable two-dimensional RPG, a live graph visualisation, persistent player state, responsive layout, multiple endings, and automated verification through a Playwright browser harness. By replacing the external API dependency with a local narrative engine, the project achieves a level of portability appropriate for academic demonstration and evaluation. These outcomes confirm that factor-graph-based decision engines can be embedded meaningfully in interactive media while remaining accessible through thoughtful visual design.

References

References

[1] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Transactions on Information Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001. doi: 10.1109/18.910572.

[2] H.-A. Loeliger, "An introduction to factor graphs," *IEEE Signal Processing Magazine*, vol. 21, no. 1, pp. 28–41, Jan. 2004. doi: 10.1109/MSP.2004.1267047.

[3] B. J. Frey, F. R. Kschischang, H.-A. Loeliger, and N. Wiberg, "Factor graphs and algorithms," in *Proc. 35th Allerton Conference on Communication, Control, and Computing*, Monticello, IL, USA, Sept. 1997, pp. 666–680.

[4] K. P. Murphy, Y. Weiss, and M. I. Jordan, "Loopy belief propagation for approximate inference: An empirical study," in *Proc. 15th Conference on Uncertainty in Artificial Intelligence (UAI)*, Stockholm, Sweden, 1999, pp. 467–475.

[5] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*. Cambridge, MA: The MIT Press, 2009. ISBN: 978-0-262-01319-2.

[6] WHATWG, "HTML Living Standard," Web Hypertext Application Technology Working Group, 2024. [Online]. Available: https://html.spec.whatwg.org/. [Accessed: Apr. 2026].

[7] W3C, "HTML Canvas 2D Context," World Wide Web Consortium, 2023. [Online]. Available: https://www.w3.org/TR/2dcontext/. [Accessed: Apr. 2026].

[8] W3C Web Audio Working Group, "Web Audio API 1.1, First Public Working Draft," World Wide Web Consortium, Nov. 2024. [Online]. Available: https://www.w3.org/TR/webaudio-1.1/. [Accessed: Apr. 2026].

[9] MDN Web Docs, "Canvas API," Mozilla Developer Network, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API. [Accessed: Apr. 2026].

[10] MDN Web Docs, "Web Audio API," Mozilla Developer Network, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API. [Accessed: Apr. 2026].